

LodeSTAR-Enhanced: Unsupervised Multi-Class Object Detection and Segmentation

Alonso Cid^{1*}

¹Facultat de Ciències, Tecnologia i Enginyeries, Universitat de Vic – Universitat Central de Catalunya (UVic-UCC), C. de la Laura,13, 08500 Vic, Spain

Associate Editor: XXXXXXXX

Received on XXXXX; revised on XXXXX; accepted on XXXXX

Keywords: Deep learning, Unsupervised learning, Geometric deep learning, Contrastive learning, Image Segmentation, Distillation

Abstract

LodeSTAR is a cutting-edge deep learning method capable of locating objects with sub-pixel accuracy from a single microscopy image. Besides performing better than other methods in complex datasets, LodeSTAR does not require large manually labelled datasets, significantly cutting down on human and computational resources. It can detect a wide range of morphological variations even if trained on a small dataset. However, its application is currently limited to the detection of a single morphological class. To extend its capabilities, we have modified its architecture through a Gumbel SoftMax layer, enabling the unsupervised semantic segmentation of objects belonging to different morphological classes. This extension inherently allows the detection of multiple classes of objects and precisely determines their positions within the image. We have successfully taught the model to classify centroid locations of objects of different classes, with similar accuracy and computational performance.

*Contact information: Alonso Cid Padilla; AlonsoCid@proton.me

1 Introduction

Computer vision is a field of computer science that aims to acquire, process, analyse, and understand digital images. Its popularity has increased since the implementation of deep learning methods, which have enhanced and automated the image analysis process [1]. Today, computers not only perform image analysis faster than humans, but they are able to perceive details and extract information that a human would not be able to.

A closely related field to computer vision in biosciences is quantitative cell imaging, which focuses on quantifying the number of cells or cellular components present in an image. Biological organisms are complex, and to understand their biological processes is necessary to analyse large quantities of data. In the field of genomics, mass sequencing techniques enabled whole genome sequencing and analysis, leading to many scientific discoveries. Similarly, microscopy image analysis used to be tedious and resource intensive, but with recent advances in microscopy technology and the automation capabilities of computer vision, the field of quantitative cell imaging has seen drastic advancements [2]. These advancements allow scientists to map, understand, and predict biological processes, making quantitative cell imaging a complementary tool for omics analyses.

Another related field that has benefited from deep learning techniques is bioimaging, which encompasses a series of techniques that allow scientists to understand biological processes by taking non-invasive images of them [3]. Since the invention of non-toxic fluorescent dyes, fluorescent microscopy have been a popular bioimaging technique in the field of microbiology. It combines optical microscopy techniques with the use of fluorescent dyes to better visualize a wide range of intracellular structures and molecules [4].

This paper is about enhancing machine learning techniques for automatic object localisation in fluorescence microscopy images. Specifically, LodeSTAR (Localisation and detection from Symmetries, Translations And Rotations), a cutting-edge method created to locate particle centroids using their geometry [5]. Before taking an in-depth look into the model, it is important

to understand the bases of machine learning and the building blocks that allowed LodeSTAR's development.

1.1 The basics of machine learning

Machine learning is a branch of artificial intelligence that focuses on developing algorithms and statistical models enabling computers to “learn” specific tasks through experience. Once these complex statistical systems are trained, they can identify patterns, extract insights and make predictions and decisions on new, previously unseen data [6]. There are four major types of machine learning algorithms: supervised, unsupervised, semi-supervised and reinforcement learning [7].

- **Supervised learning:** models are trained to learn to predict or classify a specific outcome based on a set of data, for example, classifying an object in an image or predicting the medical prognosis of a patient with cancer based on some biometric data [8]. In order to learn the selected task, it needs a collection of labelled learning data that will be used as reference to infer the output. It associates an output to a certain set of inputs [7]. The biggest limitation of this method is that it needs a previously analysed and labelled set of data, which is not always available or can be simply too time consuming.
- **Unsupervised learning:** models learn to identify patterns and structures in data without the use of labelled examples or predefined outcomes. It does not use a ground truth to know if it is predicting the desired outcome, instead, it is forced to discover hidden patterns or intrinsic structures within the input data [7]. Its ability to find patterns in seemingly stochastic data makes it a popular choice in bioinformatics [9].
- **Semi-supervised learning:** it combines the best parts of supervised and unsupervised learning. It is trained in two phases. In

the first phase, the model learns to recognise distinctive features in the data without the use of labels. During the second phase, the model is trained on a labelled dataset that allows it to use its feature detection ability to solve a specific task [10]. This method allows leveraging both labelled and unlabelled data to boost model performance [11].

- **Reinforcement learning:** The model learns to make decisions by interacting with the environment over time and receiving a reward for a positive action that leads it towards the end goal. Some techniques also apply a penalty for a negative action [12] [13].

LodeSTAR uses an unsupervised learning method called self-distillation learning, which will be explain in more detail later. This method evolved from a series of self-supervised techniques. To understand how it works is important to first understand its foundation, the techniques it builds upon, and the principles it employs.

1.2 Self-supervised learning

Self-supervised learning, similar to semi-supervised learning, is a hybrid between supervised and unsupervised learning. The difference is that even though labelled data is needed, it is not labelled by humans, but by the model itself [14]. It is important to remark that these labels are not what we understand as labels, but rather characteristics derived from the data that allow the model to discriminate among different objects. The model learns by completing a pretext task that requires the model to understand the structure of the data, like predicting a hidden part of the input or a transformation applied to the original data [15].

1.3 Self-supervised contrastive learning

In self-supervised contrastive learning methods, the input image is transformed creating two mini-batches of data, the model associates (attract) samples that come from the same transformed image and dissociates (repel) transformed

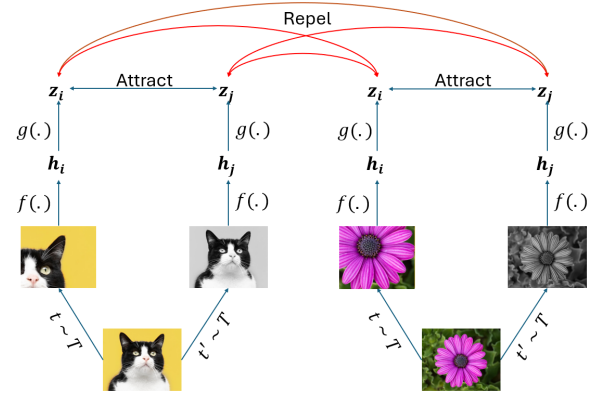


Figure 1: Architecture of a contrastive self-supervised model (specifically SimCLR). Each input image is transformed using two different augmentations, t and t' , to obtain two correlated views. The architecture uses a Convolutional Neural Network (CNN) ($f(\cdot)$) and a multilayer perceptron ($g(\cdot)$) acts as a projection head to produce a final representation (z and z'). Using a contrastive loss, the networks are encourage to produce similar representations for the two augmented views of the same image and discourage to produce similar representations from views of different images, avoiding collapse.

samples that come from different images (negative samples) (Fig.1) [16]. This allows the model to learn differential features and discern between classes [17]. To select which features the model should learn, we use transformations (cropping, rotation, translation, colour distortion and others) [14]. Each transformation limits the features a model can learn from an image, avoiding learning trivial solutions and forcing it to learn truly distinctive features. The most effective transformation combination is cropping and colour distortion, as learning common general shapes and colours is generally ineffective because these characteristics are shared by a wide variety of objects and can lead to false positives.

1.4 Self-supervised non-contrastive learning

In some cases, the model may only need to identify a single type of object, making negative samples unavailable. In that situation, self-supervised non-contrastive learning is a better and more efficient option than contrastive

learning. This approach does not require image comparisons between mini-batches and is more computationally efficient, it directly learns the features of the images [14]. One of the most famous non-contrastive learning architectures is BYOL, it uses an online (student) and a target (teacher) network. It takes a single image at a time, which is transformed twice and then passed through each network. They contain an encoder that tries to transform the transformed images into the original image. Later, a projector reduces the dimensions of the images. Finally, a predictor, unique to the online branch, transforms the representation from the online network’s projector before comparing it with the target network’s projection (Fig.2) [18]. The target network needs to be randomly initialised and use fixed weights that are updated by $\xi \leftarrow \tau \cdot \xi + (1 - \tau) \cdot \theta$, where τ is a constant, ξ is the weights of the target network and θ is the weights of the online network. The model prevents collapsing into trivial solutions by updating the target network more slowly and making both networks asymmetrical [19]. This architecture encourages the predictor’s output from the online network and the projector’s output from the target network to be as similar as possible by using a loss that measures the difference between them.

1.5 Geometric deep learning

Geometric deep learning is another important concept to understand LodeSTAR. Geometric deep learning introduces inductive biases into deep learning models to leverage notions of geometry. There are different types of geometric priors (exploits in the geometrical structure of the data) [20], symmetry and invariance being the relevant ones for LodeSTAR. By teaching a model to predict the effects of Euclidean transformations (rotations, reflections and translations), we compel it to learn the underlying geometric structure of the data. This allows the model to generalise better to unseen data, as it no longer depends on the specific orientation, position or scale of the objects within the dataset [21].

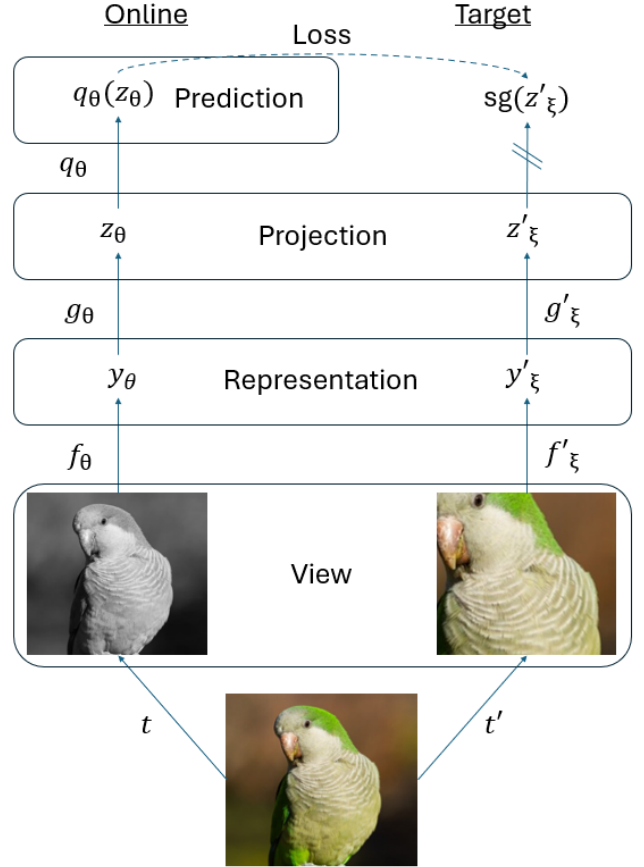


Figure 2: Representation of self-supervised non-contrastive learning model (BYOL) architecture. It calculates a similarity loss between $q_\theta(z_\theta)$ and $sg(z'_\xi)$, sg meaning stop gradient. The online network weights are represented by θ , while ξ represent the target network weights, defined as the exponential moving average of θ . The transformations are represented by t , f is an encoder, g is a projector and q is a predictor.

1.6 CNN equivariant properties

Achieving equivariance using a fully connected neural network can be very challenging, however, CNNs have innate equivariant properties allowing the network to detect similar patterns in different locations and produce feature maps that shift in response to transformations in the input (Fig.3), which saves a lot of resources during training. This is achieved through synapse weight sharing and local neural connectivity [22]. Weight sharing ensures that the same filters are applied across different regions of the input, so when an object or feature shifts in the image, its corresponding representation in the feature map shifts accordingly. Local connectivity limits the receptive field of each neuron to a small,

localised patch of the input, enabling CNNs to focus on local patterns, without regard to their position within the global structure (Fig.4).

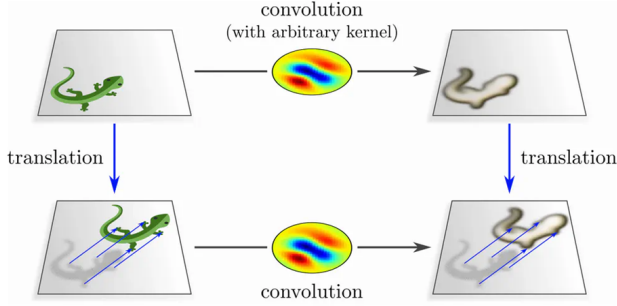


Figure 3: Representation of how translational equivariance works in CNNs. The weight map representation shift accordingly to translations in the input image. (Weiler, 2023).

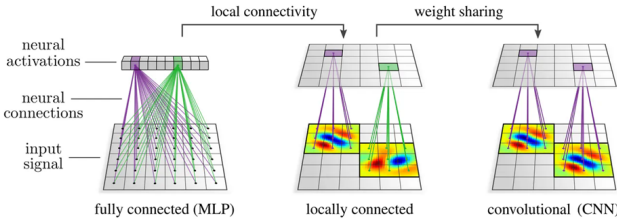


Figure 4: Representation of how local connectivity and weight sharing works in CNNs. Same filter is applied across multiple local regions. (Weiler, 2023).

1.7 Self-distillation learning

In self-distillation learning, a single neural network acts as both the teacher and student network, learning from its own predictions at earlier stages of training [23]. LodeSTAR is a self-distillation model which leverages the equivariant properties of CNNs and applies the principles of geometric deep learning. It teaches the network to identify objects by finding exact correspondence between Euclidean transformations applied to the input image, forcing the model to learn the underlying geometric properties of the objects to always find the centroid.

During training, the model takes one single image as input at a time (Fig.5a). Then, n Euclidean transformations (normally two) are applied to the image, creating n new images (Fig.5b). The neural network (it uses a CNN as

depicted in Fig.5c for its inherent equivariant properties) predicts the centroid position of the object in each image, and outputs three tensors; two with the x and y coordinates, and another with the weight map (ρ) (Fig.5d). The x and y tensors represent the direction and distance from each pixel to the object. While ρ contains the probability of finding the centre of the object in each pixel, it also determines in which pixels these calculations are needed. These three tensors form a gradient towards a point smaller than a pixel, this is how LodeSTAR accurately predicts the centre of the object (Fig.5e). Finally, the predictions are transformed back to the original image by applying the inverse Euclidean transformation, determining the centroid position in the original image (Fig.5f). If the neural network is equivariant to the transformations, both coordinates should overlap, if they don't, the difference between the predictions will be used as a loss to optimise the network [5].

LodeSTAR is an efficient model for object localisation, however, it is not able to distinguish more than one morphological class at a time. The goal of this project is to modify its architecture to give the model the capability to segment the image and classify objects that belong to different morphological classes. It is also important for us to accomplish it while maintaining the original model's training efficiency and detection accuracy.

2 Material and methods

2.1 Materials

The software used in this study was written with Python (version 3.12.5) [24] and used the following open-source libraries:

- NumPy (version 1.26.4) [25]: An essential library for performing mathematical operations with multidimensional arrays.
- PyTorch (version 2.2.2) [26]: Provides many basic tools for tensor operations and deep learning.
- SciPy (version 1.13.0) [27]: Mainly used for its module ndimage for multidimensional image processing.

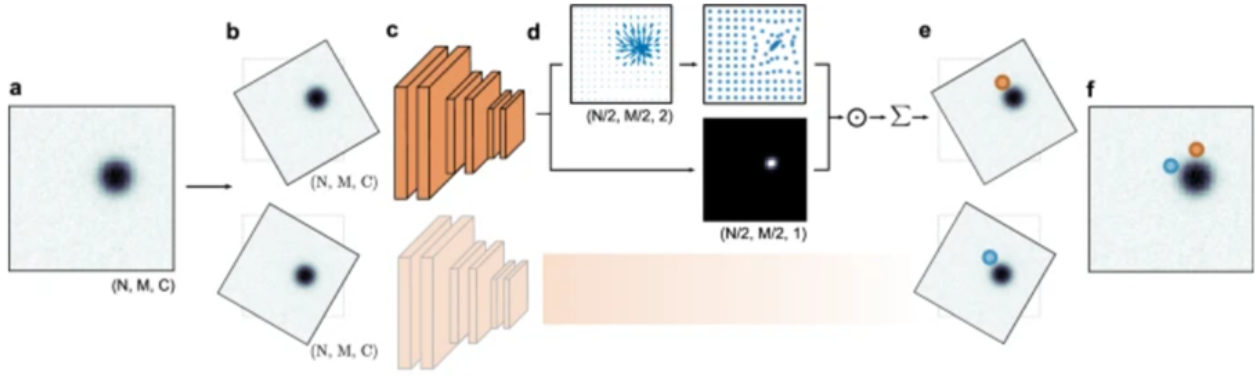


Figure 5: LodeSTAR training workflow (Midtvedt et al., 2022). **a** Input image. **b** Euclidean transformations applied to the input image. **c** CNN. **d** Output feature map and weight map of the model. **e** Centroid position prediction in the transformed images. **f** Inverse transformation applied to obtain the predicted positions in the original image for loss calculation.

- Deeplay (version 0.0.7) [28]: A library that enhances flexibility, promotes reusability and supports seamless integration of neural network components.
- Scikit-image (version 0.23.1) [29]: Provides a collection of algorithms for image processing. Used to perform morphological operations on images.
- DeepTrack (version 2.0.0rc0) [28]: Provides tools digital microscopy analysis and deep learning applications. It was used to create synthetic data.
- Matplotlib (version 3.8.4) [30]: A package for generating scientific plots for data analysis. Used to generate figures and evaluate model performance.

The hardware used was a Xiaomi Air 13.3 laptop with 8 GB of RAM, an Intel(R) Core(TM) i5-8250U CPU @ 1.60 GHz 1.80 GHz processor and Windows 11 as the operating system to perform all computational steps in our project. All the code, data and models used during this project are available in our GitHub repository (https://github.com/AlonsoCid/TFM_LodeSTAR)

2.2 Synthetic data generation

The models were trained and tested using synthetic data, generated using the DeepTrack package [28]. The simulated data resemble real fluorescent microscopy images. The data contain six types of particles or morphological

classes: circle, ellipse, quarter moon, star, filament, and square. They resemble the shapes of real particles commonly found in microscopy images (cells, organelles, protein filaments and more). Except for the square and star particles, they were created to test the model’s ability to recognise different geometries. Due to the similarities between our data and real images, the results would likely extrapolate to real data.

As mentioned previously, LodeSTAR trains on individual objects, so the training datasets comprise one hundred single-particle images per class (two hundred images for two classes and so on) (Fig.6a). The training images are 32×32 pixels in size. Additionally, the datasets include a label for each class, which was used during training to teach the model to differentiate among classes and give it the ability to correctly segment images.

For testing, multiple particles of different classes were combined in the same image. Unlike the training datasets, test datasets always contain one hundred samples, regardless of the number of classes. Only one particle per class is included in each image; multiple particles can appear in an image as long as they belong to different classes. Each test image is 96×96 pixels in size. This increase in size is necessary to avoid overlapping. The test datasets have a complex structure; each sample is composed of multiple layers as depicted in Fig.6b. Each sample in the dataset contains the simulated image that was used as input to test the model. Next, it contains one ground truth image for

every type of particle included in the dataset. Each ground truth image contains the isolated particle, eliminating even the background noise to be used during segmentation accuracy testing. Finally, the last layers contain the x and y coordinates of each particle in the dataset for detection accuracy testing.

Both datasets are a mix of the different shapes, the specific combination of particles will be explained further on. The difference in size between the training and test particles is not a problem as LodeSTAR is equivariant to size changes.

2.3 Modification to LodeSTAR CNN and architecture for image segmentation

LodeSTAR uses a CNN, originally composed of an internally handled input layer, nine convolutional hidden layers with 32, 32, 64, 64, 64, 64, 64 and 64 neurons respectively, and an output layer with the x , y and ρ channels. In order to segment, the output layer in LodeSTAR-Enhanced has been expanded by a segmentation channel (s) per number of classes (n) plus the background (β). The channel β is necessary even if we are not going to use it because it is internally treated as another class, and the model cannot learn to segment without it. To summarise, the new output layer in LodeSTAR-Enhanced is composed of x , y , ρ , s_n and β output channels.

Regarding the training process, LodeSTAR-Enhanced also trains using single-object images, although it requires them to be labelled by particle class. These labels are used as an index to indicate to the model which segmentation channel should be trained to identify each class in each iteration. This way, the model learns to recognise only one type of particle per output channel. These tensors represent the probabilities of an object of a specific morphological class being present in each pixel of the tensor. The ρ channel is no longer being optimised, and will not be used in further operations.

For s tensors to be useful for further calculations, they need to be converted into categorical tensors, containing ones in the pixels where the class of interest is located and zeros

everywhere else. However, the output layer is a continuous distribution of logit probabilities (unnormalised log probabilities), it is necessary to find a method to transform it into a binary categorical distribution. If we use some selection function like `argmax` channel-wise, it will successfully categorise each pixel but, being a non-differentiable operation (no defined slope), it will not be possible to calculate how the error propagates during backpropagation. Backpropagation is a highly efficient neural network optimisation technique, and it is essential to maintain similar performance to LodeSTAR. Thus, it is necessary to transform the segmentation tensors into a categorical distribution during the forward pass, while maintaining a continuous distribution during the backward pass to allow gradient calculation. Fortunately, this is possible using the Gumbel SoftMax trick [31] [32].

Gumbel SoftMax trick (Fig.7), also known as the categorical reparameterisation trick, uses the Gumbel and the SoftMax distribution for reparameterisation during the training of neural networks for categorisation of continuous data [33]. It takes a vector of logit probabilities, in the case of LodeSTAR-Enhanced the output of s , and adds stochastic noise in the form of random values taken from a Gumbel distribution. Then, it passes through a modified SoftMax function ($\frac{\exp(x_i/\tau)}{\sum_j \exp(x_j/\tau)}$) to select extreme values [34], classifying each value by the probability of belonging to the class of interest. The value of τ adjusts the softness of the resulting distribution: as $\tau \rightarrow 0$, the classification becomes sharper to the point of returning a discrete one-hot vector; as $\tau \rightarrow \infty$, the classification becomes softer to the point of returning a totally homogeneous distribution. LodeSTAR-Enhanced uses the PyTorch function `gumbel_softmax(hard=True)`. The function performs a double calculation; first it sets $\tau = 0$ returning a one-hot vector (`y_hard`), and then repeats the same operation with $\tau = 1$ returning a softer distribution (`y_soft`). During the forward pass it should return a one-hot vector, although the function is modified to find multiple maximum indices, allowing the classification of multiple pixels rather than only one. During the backward pass it performs the follow-

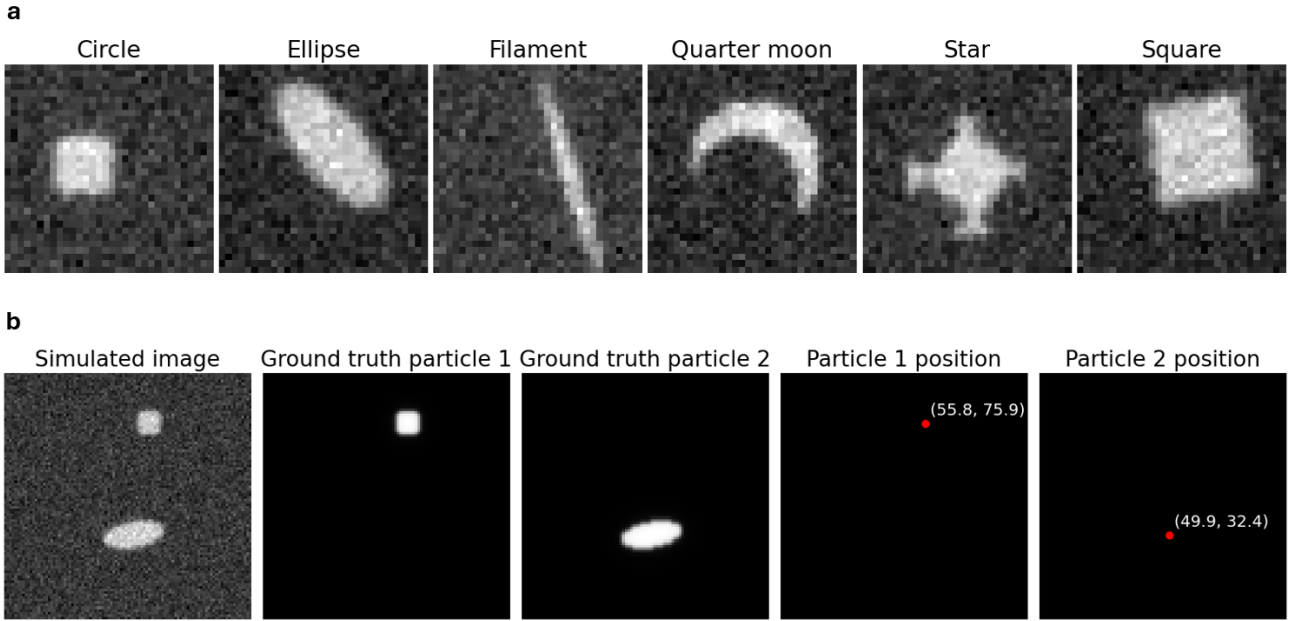


Figure 6: **a** Example of each type of particle image in the training datasets. **b** Structure of each sample in a test dataset. This dataset contains 100 images with the classes circle and ellipse. Each sample is composed of a simulated microscopy image, each segmented particle ground truth and an array with the x and y coordinates of the centroid of each particle.

ing operation `y_hard - y_soft.detach() + y_soft`, returning a decimal approximation to the `y_hard` vector, allowing the optimiser to perform gradient calculation.

2.4 Changes in LodeSTAR centroid localisation method

The original LodeSTAR uses the x, y (feature map) and ρ (weight map) to get the precise coordinates of the object centroid. It uses the feature map to check local consistency, in other words, to ensure a local maximum value is the centre of an object. This is achieved by making sure the centre is surrounded by a smooth gradient that converges at a local maximum and not a random isolated artefact, avoiding false positives. For additional consistency, the weight map is multiplied by the feature map, emphasising the true positives and suppressing any possible false positives. The result is a consistent score map that leads towards the centre of the particle. Finally, a function follows the local gradients of the new feature map that converge to a precise location, detecting the centre of each object with subpixel accuracy and returning its coordinates.

In LodeSTAR-Enhanced, the different classes

in the feature map can be located separately by using the segmentation tensors. These are multiplied by the feature map filtering off anything not related to our class, effectively creating a different feature map per type of particle. The regions that belong to our class will be multiplied by one, remaining unaffected, while the regions that do not belong to our class will be multiplied by zero, erasing them from the feature map (Fig.8). In practice, this is done by substituting the weight map for the appropriate segmentation tensor during centroid localisation calculations. This may decrease the accuracy of the detection, but the model should retain similar precision, as the weight map and the feature map should not have any significant discrepancies. The detection function now iterates over each class, using the corresponding segmentation tensor instead of the weight map.

2.5 Changes in training

To improve training stability and optimise performance, the default Deeplay training loop was replaced with a custom one that uses an exponential descending learning rate instead of a fixed one. The initial learning rate was increased from $1e-4$ to $1e-3$, as it yielded better

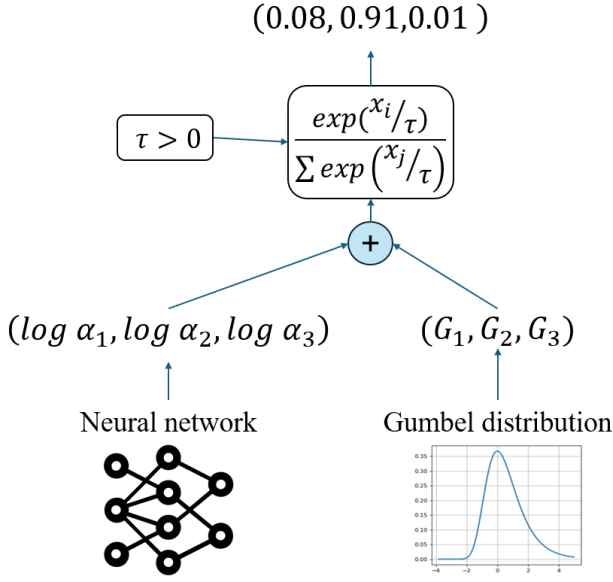


Figure 7: How the Gumbel SoftMax trick works. The neural network’s output vector of logit probabilities and a vector of random samples from a Gumbel distribution are summed to introduce stochastic noise. Then, it passes through a modified SoftMax function, which is controlled by a temperature parameter (τ). Which adjusts the smoothness of the output distribution: as $\tau \rightarrow 0$, the output becomes increasingly like a one-hot encoded vector, whilst if $\tau \rightarrow \infty$, it produces smoother, more uniform distributions.

results during testing. This approach helps the optimiser find the optimal minimum of the loss function more effectively, preventing it to oscillate around it. Additionally, the model now has the capability to save the loss in each iteration and plot it for easier quality assessment.

In certain instances, the neural network expressed inaccuracies during the training of the s channels, resulting in the appearance of minor, typically single-pixel artefacts. For this reason, a new loss function called **mask_loss** was created. These artefacts are generally present on the borders of the segmentation tensors, where finding particles is very rare. The **mask_loss** function checks all segmentation tensors to ensure that no objects are being detected in regions where finding particles is extremely unlikely. If so, they are considered inconsistencies, and the neural network is incentivised to remove them.

2.6 Performance evaluation

For model evaluation, segmentation and detection performance were tested separately to determine whether the model is able to segment the image properly whilst maintaining similar detection accuracy to the original LodeSTAR. The tests were run under different conditions using multiple datasets. We have six types of particles or classes (Fig.6a), but we limited our testing to a maximum of three different classes at a time, as more led to model instability. Additionally, each additional class increases the training dataset by one hundred images, resulting in long training times. All the tests were performed using the same datasets and the same trained models that are available on our GitHub repository.

To test segmentation performance, the output segmentation tensor of the model was compared with the ground truth image of the test dataset. This ground truth is double the size of the segmentation tensor (96×96), so it was reduced by half using a pooling operation (48×48), making them comparable. A new custom function was created (**segmentation_tests()**) to automatically perform all the analyses using the output of the model and the test dataset. First, it checks the ground truth position of the centroid in the segmentation layer; if there is a value of one in that position, it means that the object has been successfully segmented, so it counts as a true positive. Next, it checks for other groups of pixels in the image; if those groups do not match the location of a centroid, they are counted as artefacts or false positives. Finally, the Jaccard similarity index is calculated to compare how similar the segmented object is to the ground truth object by following the formula $js(A, B) = \frac{|A \cap B|}{|A \cup B|}$. The Jaccard index you will see in the results section is an average of all the Jaccard indices calculated for each image of the test dataset, the detection rate is the percentage of samples where the model has successfully segmented the object, and the false positives are the average number of artefacts found per image.

The Hungarian algorithm [35] was used to test detection accuracy. It is an optimisation algorithm that finds the most optimal solution for a task. In this case, it compares the original cen-

$$\text{Feature map} * \text{Segmentation tensor of class } n = \text{Feature map of class } n$$

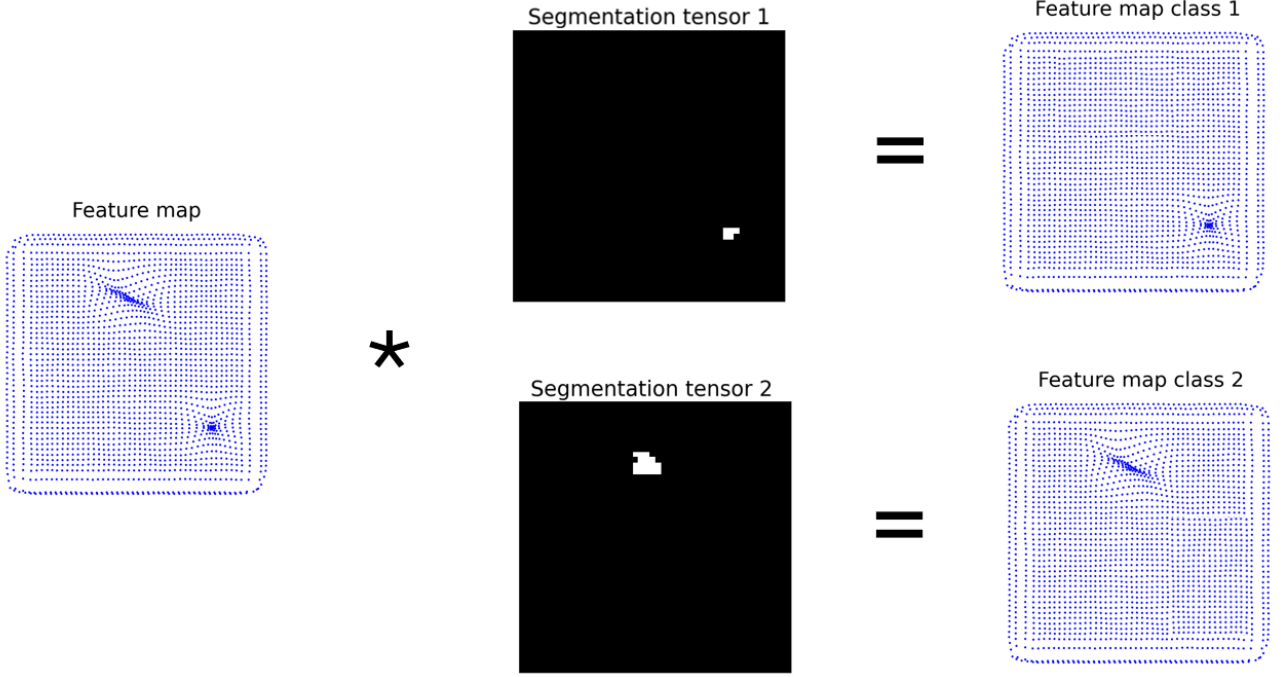


Figure 8: Representation of how a feature map is segmented in the LodeSTAR-Enhanced architecture. The features of each particle are isolated by multiplying the feature map by the segmentation tensors of each class, allowing the model to distinct them.

troid coordinates and the predicted centroid coordinates by calculating the Euclidean distance between them on a matrix and selecting the pairs that are closer together. This allows the calculation of true positives: matches between ground truth centroids and predicted centroids; false negatives: centroids without associated predictions; false positives: predicted centroids without a ground truth match; and finally, the Jaccard index, which in this case is not used to compare overlapping between tensors, but to get the proportion of true positives considering all metrics.

3 Results

3.1 Segmentation tests

Segmentation tests compare segmentation performance between different particles and multiple models trained with different numbers of particles: with one (itself), two and three classes of particles. Multiple datasets were created to represent every particle, but making every possible combination would require comparing 40

datasets, which would be impractical. Instead, all two-particle datasets contain the circle as the first particle, and all three-particle datasets contain the circle plus ellipse as the first and second particles. These shapes were chosen because they have simple geometry and very good performance, so they will add the least amount of variation to the results. The circle-ellipse dataset and circle-ellipse-square datasets (the next simplest geometry) were used to obtain circle and ellipse metrics.

Examining the segmentation performance results (Fig.9), it is difficult to discern any significant differences between classes, the only clear trend is that the star-shaped particle has a lower detection rate in a model trained for three classes (Fig.9a), which makes sense due to the shape complexity. In general LodeSTAR-Enhanced has very high detection rate on all particles. The most important differences are within classes. When comparing false positives (Fig.9b), the models trained to detect one particle produce more artefacts in the segmentation tensors. This is counterintuitive as one would expect to get more false positives as the com-

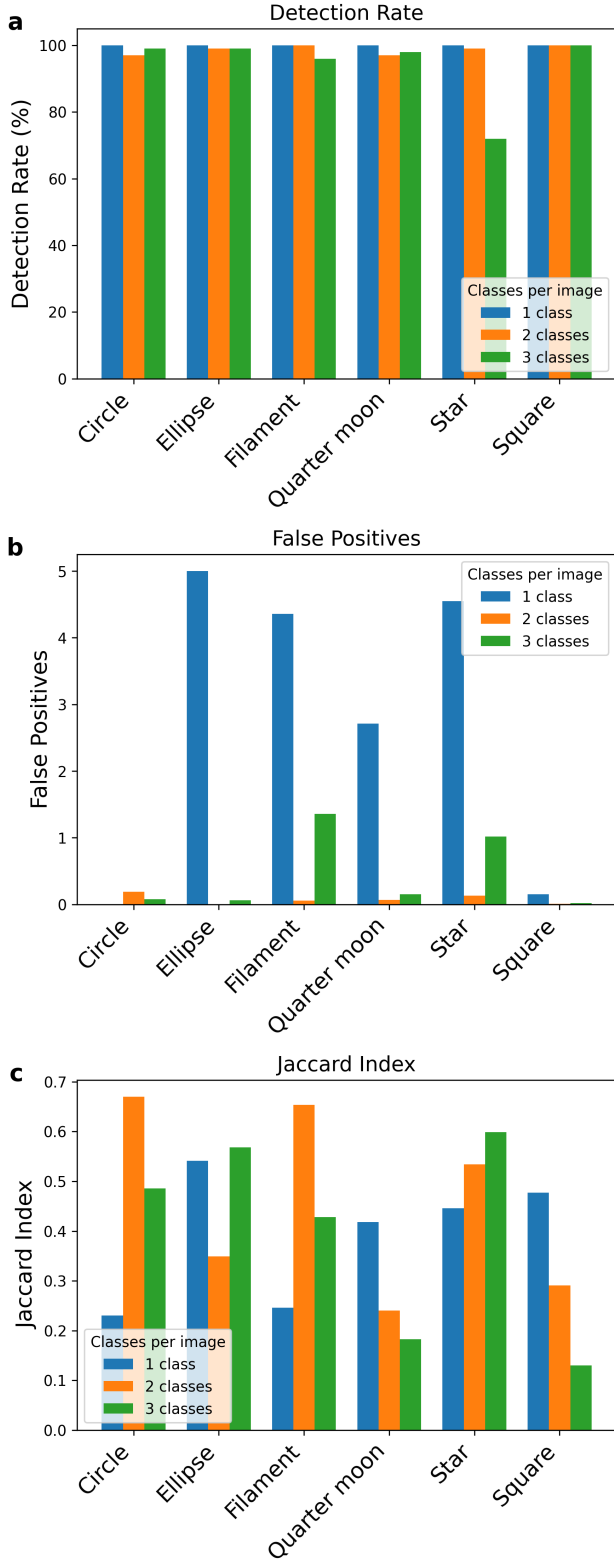


Figure 9: Each plot shows the segmentation tests results for each class from models trained to recognise 1, 2, and 3 classes. **a** The detection rate represents the percentage of correctly segmented objects. **b** The false positives represent the mean number of artefacts. **c** The Jaccard index represents the overlap between the ground truth and the segmented object.

plexity increases. The Jaccard index is inconsistent (Fig.9c) and seems to be generally lower in models trained to recognise only one particle, although this will be explained by Fig.10. It would be ideal to have perfect overlap, but it is important to consider that an overestimation in segmentation would also lower the Jaccard index but would not be nearly as detrimental for performance as an underestimation.

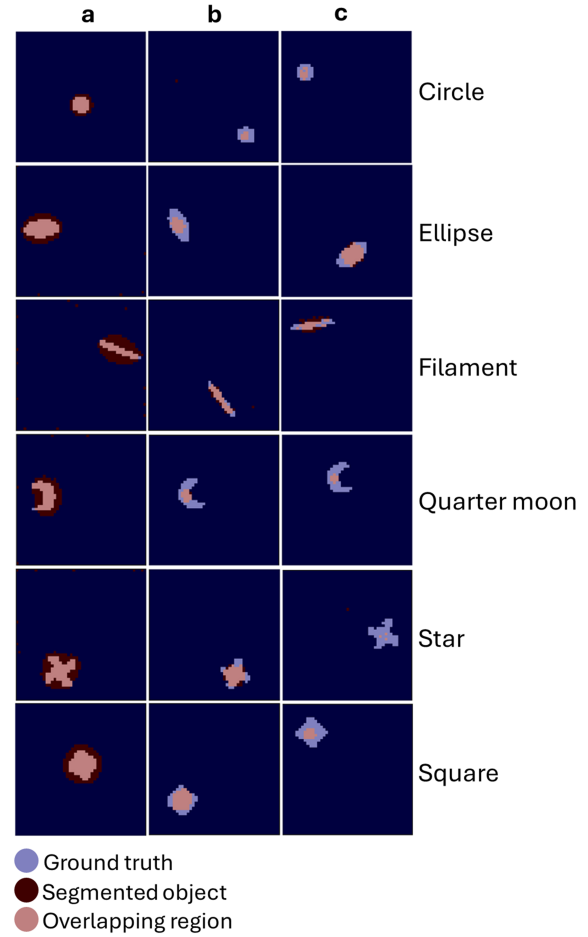


Figure 10: Super imposed segmentation tensor and ground truth image, segmentation comparison for each type of particle. **a** Images produced by a model trained to recognise only one class. **b** Images produced by a model trained to recognise two classes. **c** Images produced by a model trained to identify three classes.

Fig.10 represents the ground truth and the segmentation tensor superimposed, in blue and red respectively. The particles represented in Fig.10a, come from a model trained to detect only the particle itself. There is a clear overestimation, which explains the low Jaccard index. Although this is ideal for the model to

predict the particle centroid, as it will have a soft gradient to work with, similar to the original LodeSTAR. Models trained for two classes (Fig.10b), show an immediate reduction in segmented object size, but it consistently covers the centre and its surrounding pixels. Regarding models trained for three classes (Fig.10c), there is no significant reduction in the segmented object size, but it is less consistent. In the case of the circle, there is an empty pixel that will hinder centroid detection. In the star, only three pixels have been detected and none overlap with the centroid, which matches the poor detection shown in Fig.9b. These inconsistencies will have a negative effect on centroid detection performance.

3.2 Centroid location accuracy tests

LodeSTAR-Enhanced centroid detection accuracy has been tested against the original LodeSTAR and against LodeSTAR-Enhanced models trained with one, two and three classes to compare performance. LodeSTAR and LodeSTAR-Enhanced 1 class were trained using circular particles; LodeSTAR-Enhanced 2 classes was trained using circular and elliptical particles; and LodeSTAR-Enhanced 3 classes was trained using circular, elliptical and square particles.

Comparing LodeSTAR-Enhanced 1 class with LodeSTAR (Fig.11), it is noticeable that the median distance between predicted and ground truth centroids is higher in LodeSTAR-Enhanced and there are two outliers. A Wilcoxon test for paired data confirms that there are significant differences between the two ($p < 0.05$). Even though this represents a slight downgrade in performance, the model remains highly precise. When increasing the number of classes, the accuracy of the model remains stable, but there is an increasing number of outliers, most likely a consequence of the previously mentioned segmentation problems.

Looking at the results of the Hungarian algorithm (Table.1), it shows that LodeSTAR and LodeSTAR-Enhanced 1 class have similar performance. They do not produce any false negatives, always predict the centroid of every object, and do not have any false positives ei-

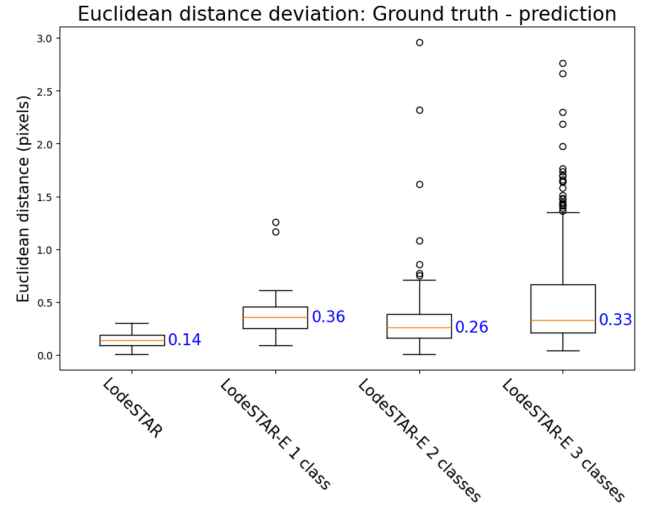


Figure 11: Comparison of centroid prediction accuracy across multiple models, LodeSTAR-Enhanced being represented as LodeSTAR-E for simplicity. The y-axis represents the Euclidean distance between ground truth centroids and predicted centroids. The value in blue within each box represents the median.

ther. When trained to detect more than one class, false positives start to appear and subsequently the Jaccard index decreases.

By looking at the test images in Fig.12, it is clear that the difference in performance between LodeSTAR and LodeSTAR-Enhanced is indistinguishable at first glance. However, on the elliptical object in the last image, it is possible to perceive one of the previously seen outliers, the predicted centroid position is visibly off.

4 Discussion and Conclusion

The objective of this thesis was to expand LodeSTAR's capabilities by giving it a completely new property: image segmentation. This has been an especially challenging task considering that the model is only able to recognise geometrical shapes and lacks the ability to identify complex details. This is why the segmented objects tend to be circular or elliptical. Even though there is still significant room for improvement, given the challenges in segmentation and the tight schedule, we consider LodeSTAR-Enhanced to be a success.

A clear next step will be to enhance the model's segmentation capabilities. More consistent object segmentation will likely resolve

Model	True positives	False negatives	False positives	Jaccard index
LodeSTAR	100	0	0	1
LodeSTAR-E 1 class	100	0	0	1
LodeSTAR-E 2 classes	200	0	10	0.952
LodeSTAR-E 3 classes	300	0	20	0.937

Table 1: Performance statistics obtained using the Hungarian method for different models and conditions. LodeSTAR-Enhanced is represented as LodeSTAR-E.

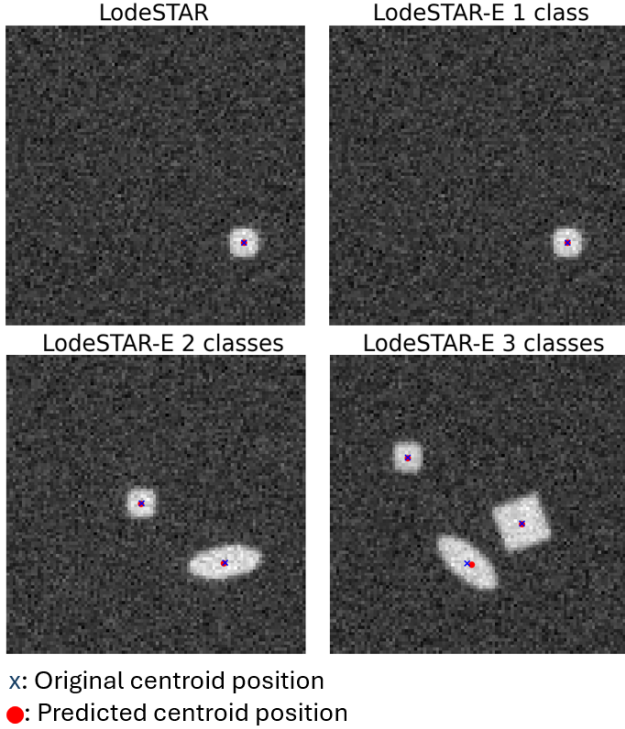


Figure 12: The image shows ground truth and predicted centroid positions over their test images for LodeSTAR and LodeSTAR-Enhanced (LodeSTAR-E) trained with multiple classes.

the remaining detection issues. Although the model’s stability has been greatly improved by the implementation of an exponential learning rate, there is still room for improvement. Whilst uncommon, it remains possible to end up with a poorly optimised model after training, indicating that further work on the training process could result in additional significant improvements.

Due to time limitations, several important tests were left out, which would be necessary to fully assess real-life performance. Firstly, we consider crucial to re-run the tests using real fluorescence microscopy images, as simulated images lack the complexity and variability of real ones, so real performance is unknown. An-

other important test is multiple object detection of same-class particles. While we know that the original LodeSTAR is very capable of locating multiple objects, detection now depends on the segmenting capabilities of the model, so it is not possible to predict its performance. Finally, a detection resolution test would be necessary to determine the minimum distance between two objects for the model to recognise them as distinct entities. Our data have been generated to avoid overlapping by ensuring a minimum distance of fifteen pixels between each particle, so performance beyond that threshold is unknown.

As technology advances and microscopy methods become more efficient, it has become easier and cheaper to collect data. This increase in data production requires a corresponding increase in analysis capabilities. Currently, image analysis is one of the major bottlenecks in bio-science research, and the demand for AI models that can alleviate this problem is rapidly increasing. LodeSTAR is likely the most efficient model for particle localisation, and by giving it the ability to differentiate between types of particles, we have significantly increased its usefulness.

Even though there are still many challenges before LodeSTAR-Enhanced can be considered ready for real-world applications, this project has been a big step forward. The development of LodeSTAR-Enhanced represents a significant advancement in addressing the growing demand for efficient image analysis tools in bioscience research. As it continues to evolve, it could potentially play a crucial role in many automated microscopy image analysis pipelines, substantially accelerating research processes and reducing costs.

5 Acknowledgments

I would like to thank my supervisor, Dr. Carlo Manzo, and co-supervisors, Jesus Pineda and Mirja Granfors, for their guidance. I'm also grateful to Benjamin Midvedt and Dr. Giovanni Volpe for their valuable assistance.

This project was partly supported through grant PID2021-125386NB-I00 funded by MCIN/AEI/10.13039/501100011033/ and “ERDF A way of making Europe”

References

- [1] Shuyuan Xu, Jun Wang, Wenchi Shou, Tuan Ngo, Abdul-Manan Sadick, and Xiangyu Wang. Computer vision techniques in construction: a critical review. *Archives of Computational Methods in Engineering*, 28:3383–3397, 2021.
- [2] Neda Bagheri, Anne E Carpenter, Emma Lundberg, Anne L Plant, and Rick Horwitz. The new era of quantitative cell imaging—challenges and opportunities. *Molecular cell*, 82(2):241–247, 2022.
- [3] Harsh S Lahoti and Sangita D Jogdand. Bioimaging: evolution, significance, and deficit. *Cureus*, 14(9), 2022.
- [4] Shane M Hickey, Ben Ung, Christie Bader, Robert Brooks, Joanna Lazniewska, Ian RD Johnson, Alexandra Sorvina, Jessica Logan, Carmela Martini, Courtney R Moore, et al. Fluorescence microscopy—an outline of hardware, biological handling, and fluorophore considerations. *Cells*, 11(1):35, 2021.
- [5] Benjamin Midtvedt, Jesús Pineda, Fredrik Skärberg, Erik Olsén, Harshith Bachimanchi, Emelie Wesén, Elin K Esbjörner, Erik Selander, Fredrik Höök, Daniel Midtvedt, et al. Single-shot self-supervised object detection in microscopy. *Nature communications*, 13(1):7492, 2022.
- [6] Reinaldo Padilha França, Ana Carolina Borges Monteiro, Rangel Arthur, and Yuzo Iano. An overview of deep learning in big data, image, and signal processing in the modern digital age. *Trends in deep learning methodologies*, pages 63–87, 2021.
- [7] I. H. Sarker. Machine learning: Algorithms, real-world applications and research directions. *SN Computer Science*, 2(160), 2021.
- [8] C. M. Lynch, B. Abdollahi, J. D. Fuqua, A. R. de Carlo, J. A. Bartholomai, R. N. Balgeman, V. H. van Berkel, and H. B. Frieboes. Prediction of lung cancer patient survival via supervised machine learning classification techniques. *International Journal of Medical Informatics*, 108:1–8, 2017.
- [9] Zoubin Ghahramani. Unsupervised learning. In *Summer school on machine learning*, pages 72–112. Springer, 2003.
- [10] Yanbei Chen, Massimiliano Mancini, Xiatian Zhu, and Zeynep Akata. Semi-supervised and unsupervised deep visual learning: A survey. *IEEE transactions on pattern analysis and machine intelligence*, 46(3):1327–1347, 2022.
- [11] Tri Huynh, Aiden Nibali, and Zhen He. Semi-supervised learning for medical image classification using imbalanced training data. *Computer methods and programs in biomedicine*, 216:106628, 2022.
- [12] Yuxi Li. Deep reinforcement learning: An overview. *arXiv preprint arXiv:1701.07274*, 2017.
- [13] V Mnih. Playing atari with deep reinforcement learning. *arXiv preprint arXiv:1312.5602*, 2013.
- [14] Veenu Rani, Syed Tufael Nabi, Munish Kumar, Ajay Mittal, and Krishan Kumar. Self-supervised learning: A succinct review. *Archives of Computational Methods in Engineering*, 30(4):2761–2775, 2023.
- [15] Kriti Ohri and Mukesh Kumar. Review on self-supervised image recognition using deep neural networks. *Knowledge-Based Systems*, 224:107090, 2021.
- [16] Ting Chen, Simon Kornblith, Mohammad Norouzi, and Geoffrey Hinton. A simple framework for contrastive learning of visual representations. In *International conference on machine learning*, pages 1597–1607. PMLR, 2020.
- [17] Rayan Krishnan, Pranav Rajpurkar, and Eric J Topol. Self-supervised learning in medicine and healthcare. *Nature Biomedical Engineering*, 6(12):1346–1352, 2022.
- [18] Jean-Bastien Grill, Florian Strub, Florent Altché, Corentin Tallec, Pierre Richemond, Elena Buchatskaya, Carl Doersch, Bernardo Avila Pires, Zhaohan Guo, Mohammad Gheshlaghi Azar, et al. Bootstrap your own latent—a new approach to self-supervised learning. *Advances in neural information processing systems*, 33:21271–21284, 2020.
- [19] Y. Tian, X. Chen, and S. Ganguli. Demystifying a key self-supervised learning technique: Non-contrastive learning. <https://ai.meta.com/blog/demystifying-a-key-self-supervised-learning-technique-non-contrastive-learning/>, July 7 2021. Meta AI Blog.
- [20] Michael M Bronstein, Joan Bruna, Taco Cohen, and Petar Veličković. Geometric deep learning: Grids, groups, graphs, geodesics, and gauges. *arXiv preprint arXiv:2104.13478*, 2021.
- [21] J. Tarbouriech. A brief introduction to geometric deep learning. <https://towardsdatascience.com/a-brief-introduction-to-geometric->

- deep-learning-dae114923ddb, June 23 2019. Towards Data Science.
- [22] M. Weiler. Convolutional networks & translation equivariance. https://mauriceweiler.gitlab.io/blog_post/cnn-book_2_conventional_cnns/, November 21 2023. Maurice Weiler’s Blog.
 - [23] Linfeng Zhang, Jiebo Song, Anni Gao, Jingwei Chen, Chenglong Bao, and Kaisheng Ma. Be your own teacher: Improve the performance of convolutional neural networks via self distillation. In *Proceedings of the IEEE/CVF international conference on computer vision*, pages 3713–3722, 2019.
 - [24] Guido Van Rossum and Fred L Drake. *Introduction to python 3: python documentation manual part 1*. CreateSpace, 2009.
 - [25] Charles R Harris, K Jarrod Millman, Stéfan J Van Der Walt, Ralf Gommers, Pauli Virtanen, David Cournapeau, Eric Wieser, Julian Taylor, Sebastian Berg, Nathaniel J Smith, et al. Array programming with numpy. *Nature*, 585(7825):357–362, 2020.
 - [26] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, et al. Pytorch: An imperative style, high-performance deep learning library. *Advances in neural information processing systems*, 32, 2019.
 - [27] Pauli Virtanen, Ralf Gommers, Travis E Oliphant, Matt Haberland, Tyler Reddy, David Cournapeau, Evgeni Burovski, Pearu Peterson, Warren Weckesser, Jonathan Bright, et al. Scipy 1.0: fundamental algorithms for scientific computing in python. *Nature methods*, 17(3):261–272, 2020.
 - [28] Benjamin Midtvedt, Saga Helgadottir, Aykut Argun, Jesús Pineda, Daniel Midtvedt, and Giovanni Volpe. Quantitative digital microscopy with deep learning. *Applied Physics Reviews*, 8(1), 2021.
 - [29] Stefan Van der Walt, Johannes L Schönberger, Juan Nunez-Iglesias, François Boulogne, Joshua D Warner, Neil Yager, Emmanuelle Gouillart, and Tony Yu. scikit-image: image processing in python. *PeerJ*, 2:e453, 2014.
 - [30] John D Hunter. Matplotlib: A 2d graphics environment. *Computing in science & engineering*, 9(03):90–95, 2007.
 - [31] E. Benjaminson. Gumbel-softmax. <https://sassafra13.github.io/GumbelSoftmax/>, August 13 2020. GitHub Pages.
 - [32] W. Wong. What is gumbel softmax? <https://towardsdatascience.com/what-is-gumbel-softmax-7f6d9cdcb90e>, May 17 2020. Towards Data Science.
 - [33] Eric Jang, Shixiang Gu, and Ben Poole. Categorical reparameterization with gumbel-softmax. *arXiv preprint arXiv:1611.01144*, 2016.
 - [34] Chris J Maddison, Andriy Mnih, and Yee Whye Teh. The concrete distribution: A continuous relaxation of discrete random variables. *arXiv preprint arXiv:1611.00712*, 2016.
 - [35] Harold W Kuhn. The hungarian method for the assignment problem. *Naval research logistics quarterly*, 2(1-2):83–97, 1955.